PROJECT 2

Write Up

By Keanu Arao

Introduction

This project is a C++ recreation of the board game "Life." The main influence for this project is due to it being a project for my CIS/CSC 17A class and it being my favorite board game. This is actually the 2nd project of the class and is a continuation of the first one. This project focuses more on C++ classes and how to implement different, complicated concepts about them. The main difference this C++ version will have compared to the board game, is that there are no split pathways after stops; everything just continues off one path—this is for program simplicity. This project has a lot of importance for me as it has provided me with a lot of fun and experience in programming in C++ and in being a more advanced programmer. This project alone has helped me learn about different concepts and implementation of classes and due to it forcing me to adapt quickly, I have gained a stronger understanding of these concepts. I want to be able to use this project later in the future as a reference to my learning journey and be able to create a more complex version of this program where aspects of the game may be more appealing, interactable, and enjoyable. Overall, this project will be a big learning curve in my journey to learn programming.

Summary

The main file of the game is "GameOfLifeV2.cpp." This file contains about 700 lines of code that pieces together all of the functions, class implementations, and file readings. This main file is also supported by "GameOfLifeV2_SecondaryFile.cpp," which contains over 200 lines of code and is used to create object arrays of the classes implemented and write them to binary files so that the main program can read from them. The secondary file is necessary as it sets up all the

binary file data for the main game, which means that it makes the game more customizable. This concept alone can be used to make it so that players can customize their games for a more enjoyable or challenging experience. Other important files in this project are the header files which contain their respective classes. Some of these files are “Card.h,” which provides the “Card” class which sets up a blueprint of how Card ownership will be determined, and “Player.h” which creates a “Player object” that is used to track the player’s stats and use certain member functions to help the player. With the main, secondary, and header files, altogether, they add up to over 1200+ lines of code, which means that they should pass the 1000-line minimum requirement. Another requirement that is met for this project is the use of concepts from chapters 13-16 of the Gaddis Textbook. The information to check whether these requirements are met will be shown in the “Concepts” section of this write-up. In terms of material that was used outside of the required concepts for classes, it could be said that there were many uses of if statements, loops, array searching, and enum structures.

Overall, this project took about a few (2-3) weeks to complete as it was shortly assigned after the end of project 1 (the previous version of this project). Even though the time schedule was tight, I wanted to make sure that I understood most, if not all, of the material to the best of my ability. The project was difficult at first, but gradually as I learned and studied the concepts needed to fulfill the requirements, progress became much easier. Even if this project doesn’t receive a grade that I will be satisfied with, I will be glad to accept that this project has taught me a lot about programming and the complexity of object-oriented programming.

This game will be considered “finished” for the sake of this project, but in my mind, it will still be counted as “in-progress.” This is due to me wanting to expand this project into something bigger; to reuse this project over again with new learning curves to understand more complex programming concepts. I am still missing aspects of the game that I wanted to

implement, but those will be much more complicated and seem like topics for me to study later. In likeliness, I will still be working on this project for passion/leisure in the future.

Description & How the Solution was Programmed

Here are the main rules, functions, and description of the game.

The game "Life" is a board game that imitates a fun, hectic, and weird journey through the actual life of a person. The main game can have up to four players, who are competing to get the most amount of money in the end. Each player starts out with $200k and works their way up (or across) the board experiencing different aspects of life such as marriage, college, getting a house, or having children. All these aspects will affect your wealth at the end of the game when you "retire" in life. At the beginning of the game, each player chooses whether to take the "College Career Path" or the regular "Career Path." In the actual board game, this affects your starting position on the board as picking the college career path will give you a longer route as you need time to graduate college. However, in this C++ version, this function is not programmed for simplicity, so this decision may be unfair for now as picking the college career path will likely give you a higher paying career. Another function in the board game that is not implemented in this C++ version is the concept of loans. In the board game, when a player's money goes below zero, they can apply for a loan, which they will need to pay off when they retire. This function is not set in this program for complex reasons and will only set your money to zero. There are a few more details to learn about, but you only necessarily control spinning for much you get to move, so the program does everything else for you (i.e. paying for a house, swapping career cards, etc.). It's somewhat kind of like a simulation, but you get to interact with it.

The main issue with this project was trying to figure out how to “convert” the structures to classes and implementing the concepts from chapters 13-16 of the Gaddis textbook into my program. To get a kickstart, I first started writing out basic blueprints for the more important classes, such as the Player and Card classes. Once I had these down, I could then convert the Career, House, and Action card structures into their own classes. I even was able to implement a new class called Pet, which would be a card given to a player and have a chance to do something random each turn. Once I had everything converted, I then made it so that the classes that represented the cards in the board game derive from the Card class. For example, this made it easier to get a stronger understanding of how I can implement the relationship of how House and Action classes are cards. They are both cards, but they have different uses, but they can both be owned or placed on a deck. When I was able to grasp the relationships between the cards down, it made it much easier to program the member data and functions to tie into each other and to programs outside the header files.

From there, I was able to slowly implement certain concepts into the project. For example, if I wanted to share a “display” function for all the card classes, I could polymorph a virtual function from the actual Card class so that it would uniquely display information about each card individually. The main process of implementing features was simply researching a concept, brainstorming how it would affect my game, then testing it out multiple times to see if it worked and if I liked it or not.

Sample Input/Output

The first section where you would have to input is when the program asks for how many players. To continue to the game, you must enter a number from 1 to 4. If an incorrect input is received, it will re-prompt the user.

Input: 2 (correct)

```
Enter the amount of players you want playing (Max of 4 players):2
Enter player 1's name:
```

Input: 5 (incorrect)

```
Enter the amount of players you want playing (Max of 4 players):5
Invalid input. Enter a number (1-4)
>
```

The next inputs you will then have to enter after the player count, are the player's name, decision for college, and decision for career. This will loop until the information for all the players is given.

Inputs in order: 2, "player1", y, 2

```
Enter the amount of players you want playing (Max of 4 players):2
Enter player 1's name: player1
Would you like to go to college? (Y/N): y
========================================
Pick one of the two careers:
Job 1: Nurse
Salary: 80k
Job 2: Pharmacist
Salary: 90k
Enter 1 or 2 in accordance to the job:
2
You have chosen: Pharmacist
========================================
Enter player 2's name:
```

This input is when a player lands on a house spot.

Input: 1

```
========================================
p2's turn!
Tap enter to spin:
You rolled a(n) 10
You have passed a pay position!
You now have 570K
H
========================================
Please select the House you want to buy.
(Enter 0 to cancel)
House 1: Penthouse Suite
Cost: 410
Resell 1: 390
Resell 2: 490
----------------------------------------
House 2: Modern Condo
Cost: 300
Resell 1: 290
Resell 2: 320
>1
You bought the Penthouse Suite!
========================================
```

Flowcharts/Pseudocode/UML

This section focuses on the core flowchart and pseudo-code made for the projects. These outline the main idea of the programs and their functions.

THE PSUEDO CODE

BEGIN

SET random seed using current time

DECLARE variables:

jSize ← 20

hSize ← 20

bSize ← 100

aSize ← 70

petList ← 10

pCount ← player count

Create arrays for careers, college careers, actions, houses, pets, and board

Initialize enums: StpType and PayPos

TRY TO:

READ data from files into:

- CollegeCareers.dat → cc
- Careers.dat → careers
- Houses.dat → houses
- board.dat → board
- Pets.dat → pets
- ActionCards.dat → actions

IF reading fails

DISPLAY error and exit

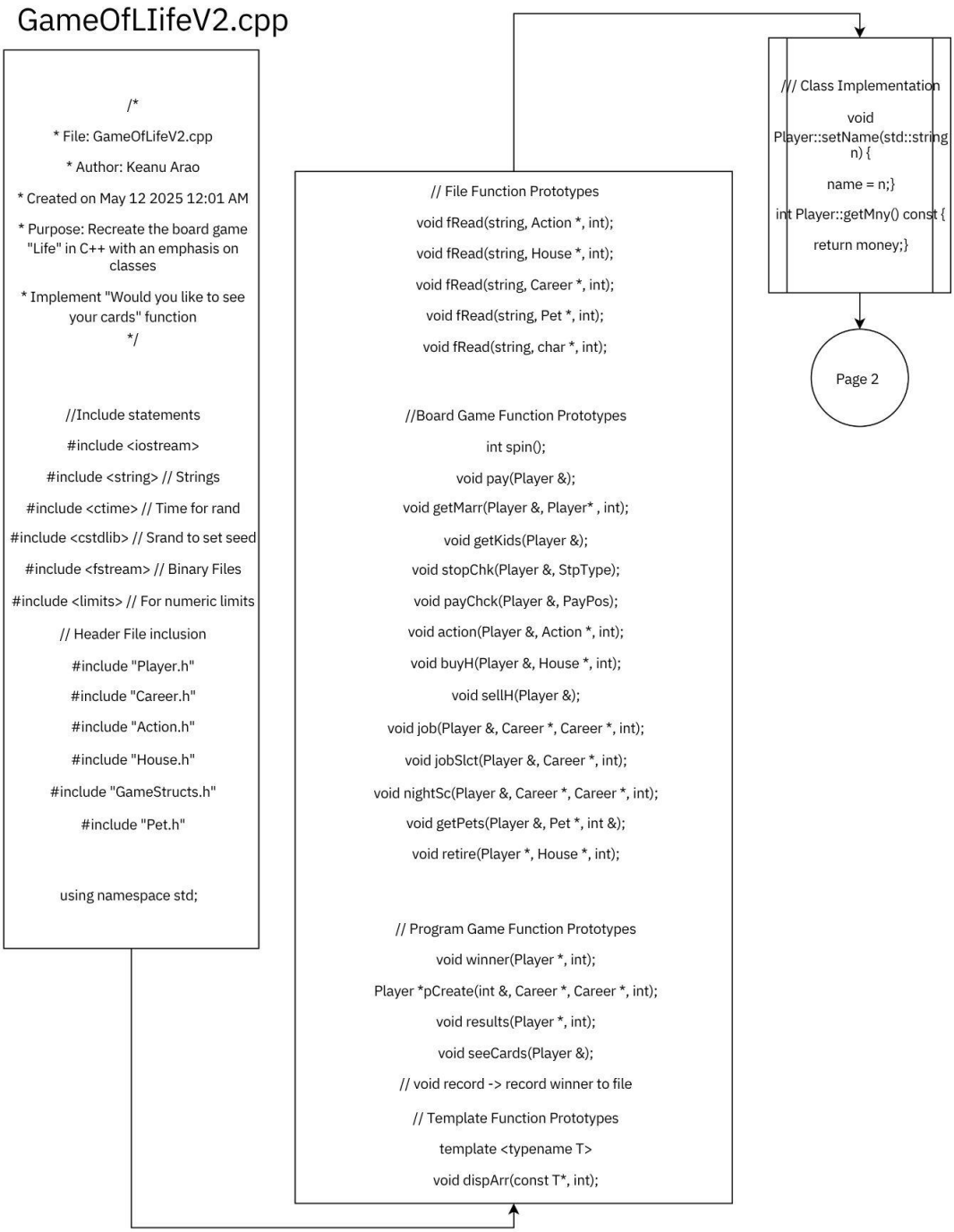
```
PROMPT user: "Enter the amount of players (Max 4):"

LOOP UNTIL valid input:

   IF input invalid (not in 1–4)

      PROMPT again



CREATE players using pCreate()



DISPLAY "Game starting..."



REPEAT until all players reach end:

   SET eCount ← 0



   FOR each player in players:

      IF player has reached final board space:

         INCREMENT eCount

         SKIP to next player



      DISPLAY current player's turn
```

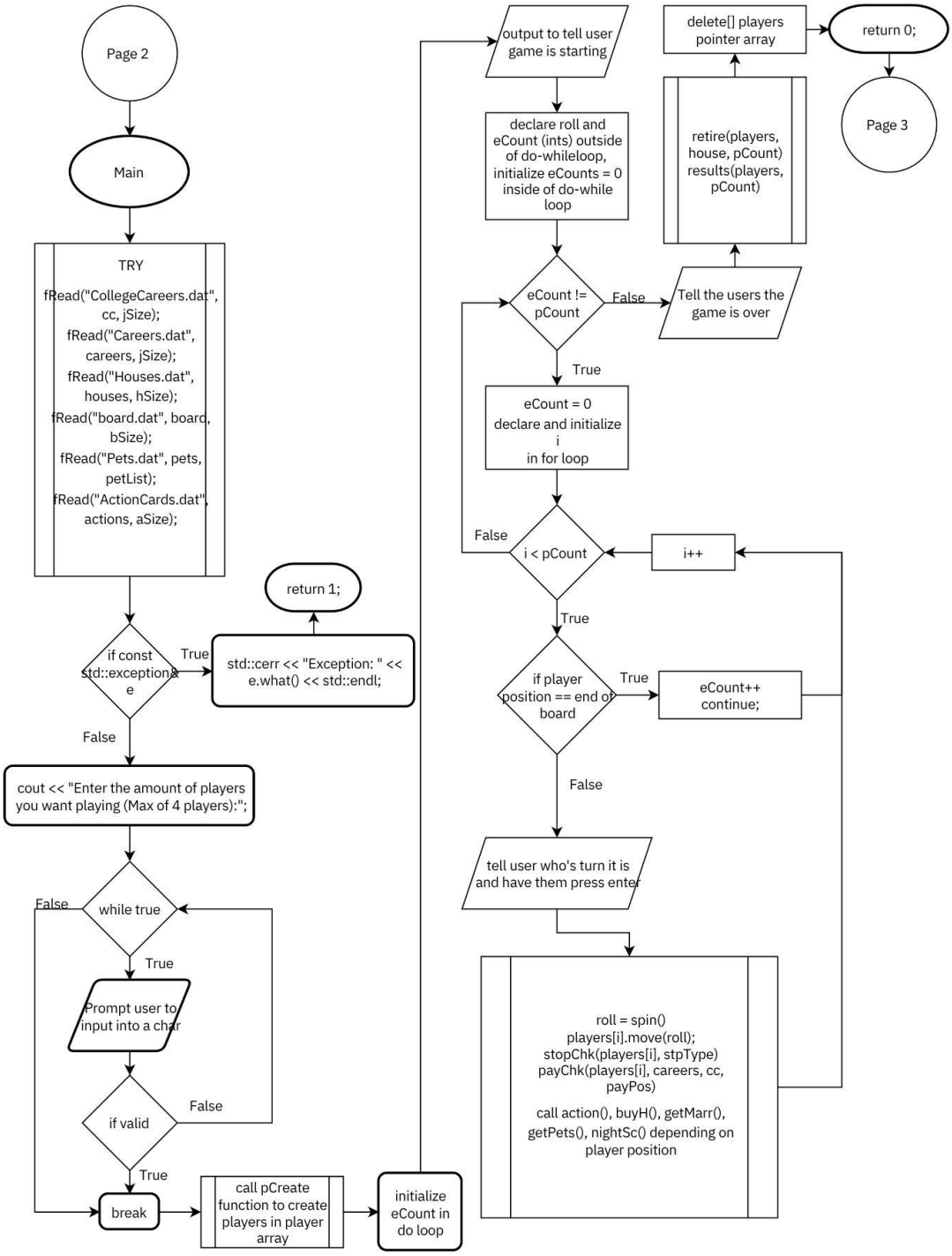
```
PROMPT user to press enter to spin


roll ← spin()

DISPLAY roll result


MOVE player forward by roll amount


CALL stopChk() to check for stop space

CALL payChck() to check for pay space


GET current board space character


SWITCH board space character:

   CASE 'A':

      CALL action() with Action deck

   CASE 'P':

      CONTINUE (Pay space already handled)

   CASE 'V':
```

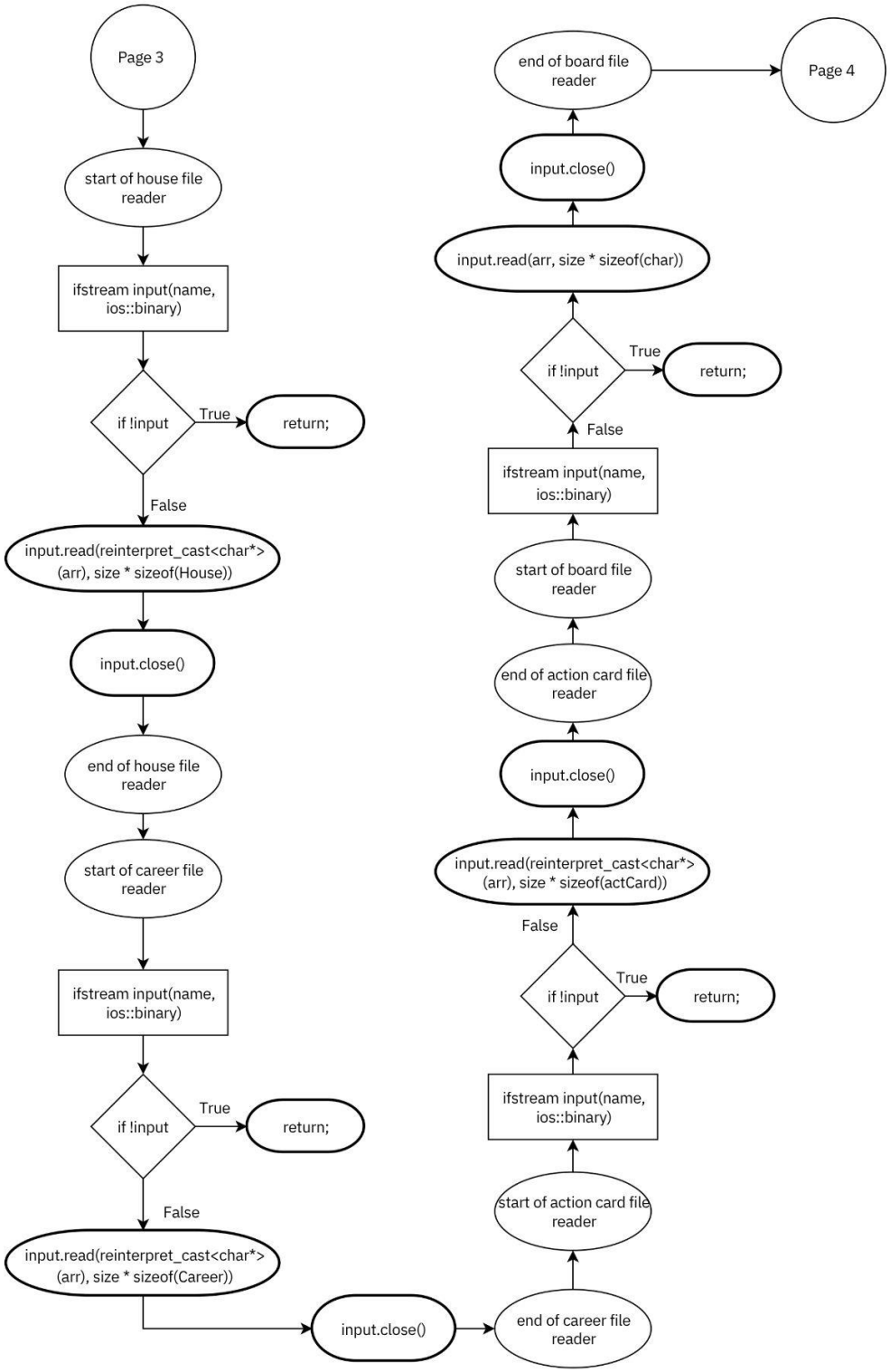
```
        DISPLAY vacation bonus

        ADD vacation bonus to player

    CASE 'H':

        CALL buyH() to buy a house

    CASE 'M':

        CALL getMarr() to get married

    CASE 'C':

        CALL getPets() to adopt pets

    CASE 'K':

        CALL getKids() to add kids

    CASE 'N':

        CALL nightSc() for night school job change

    CASE 'R':

        DISPLAY "You are retired"

        CONTINUE

    DEFAULT:

        DISPLAY "Error"


UNTIL eCount == total number of players
```

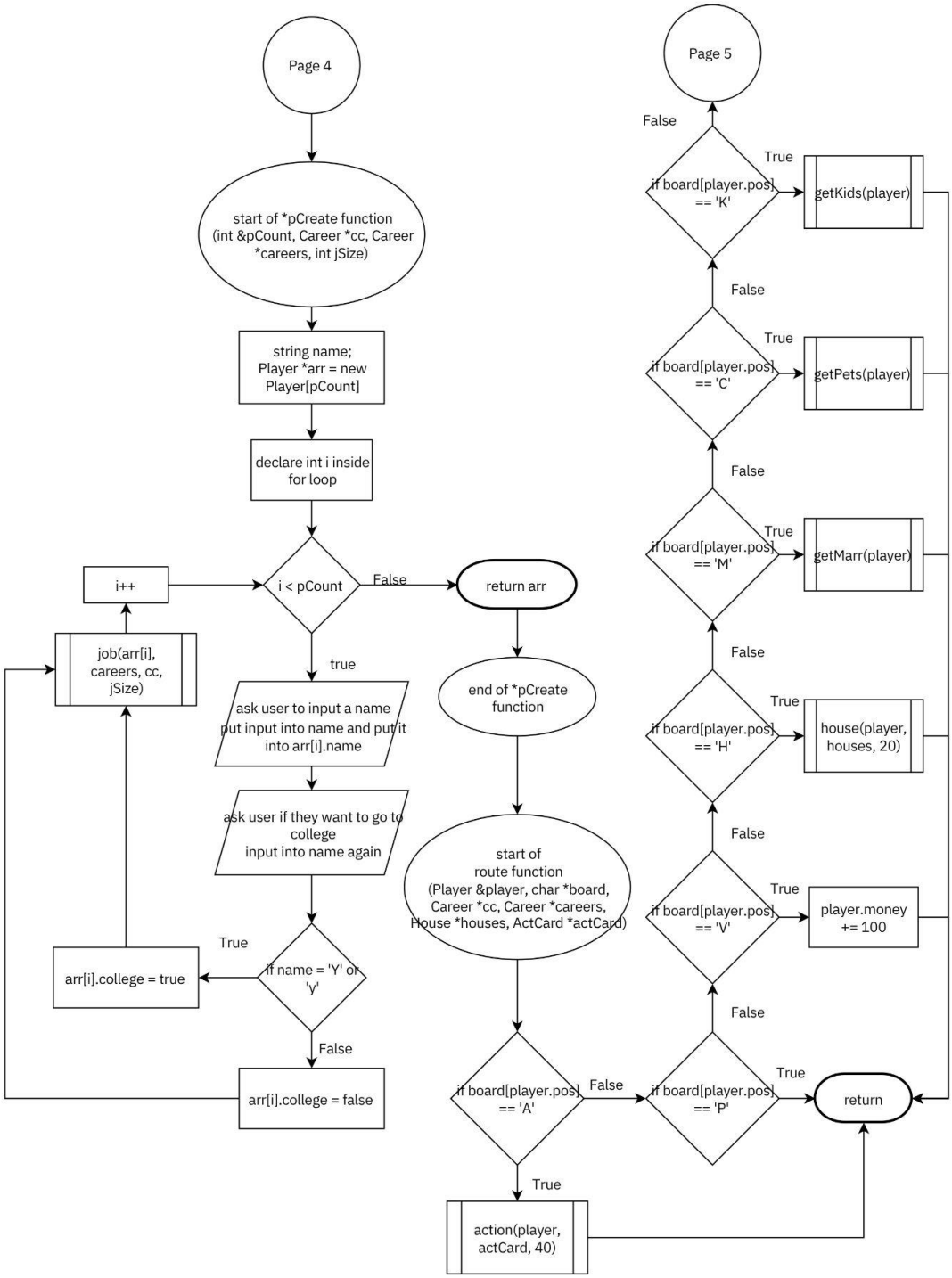
```
    CALL retire() to finalize endgame scores

    CALL results() to show final game summary



    DEALLOCATE players array

END
```

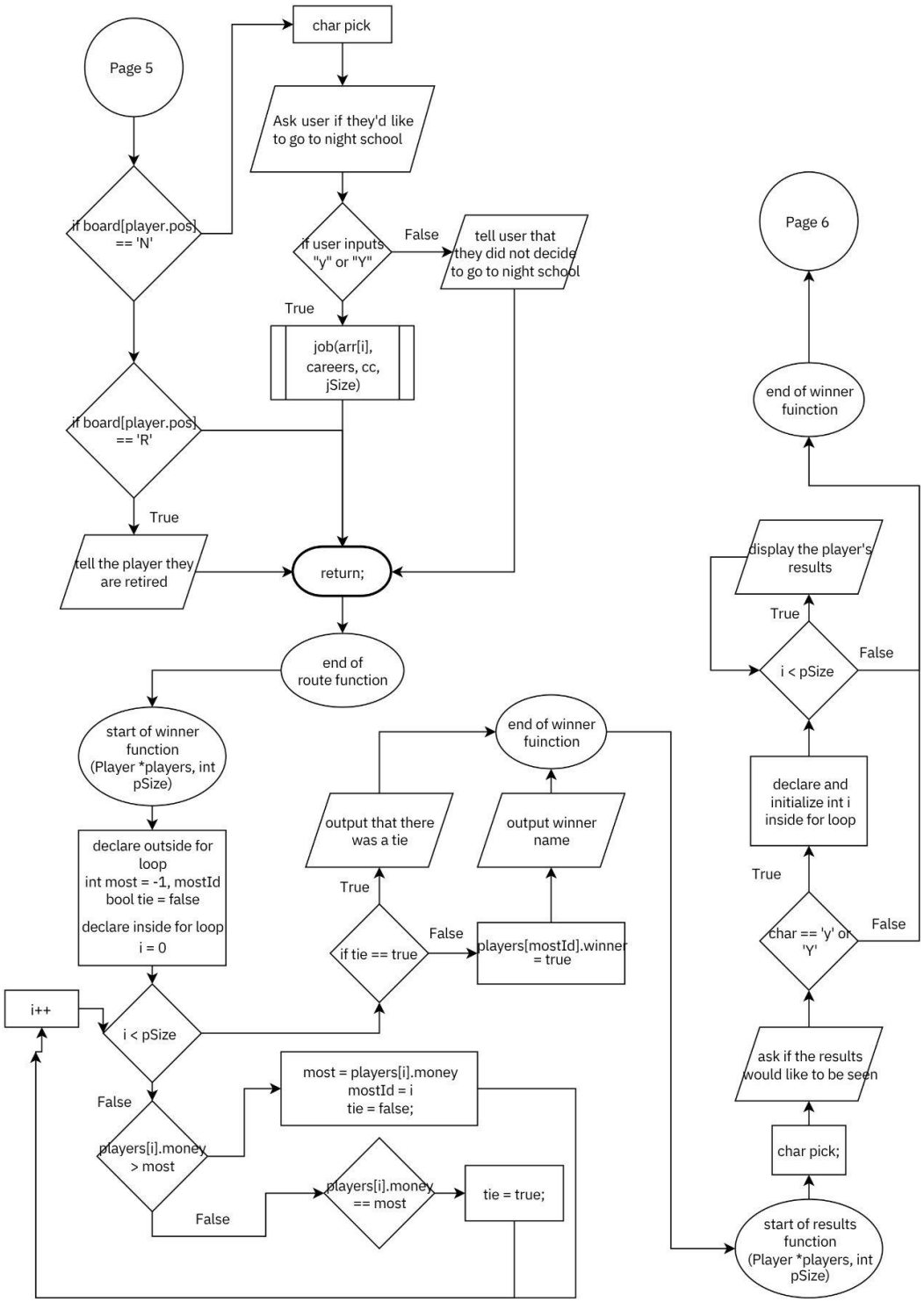
FLOWCHART

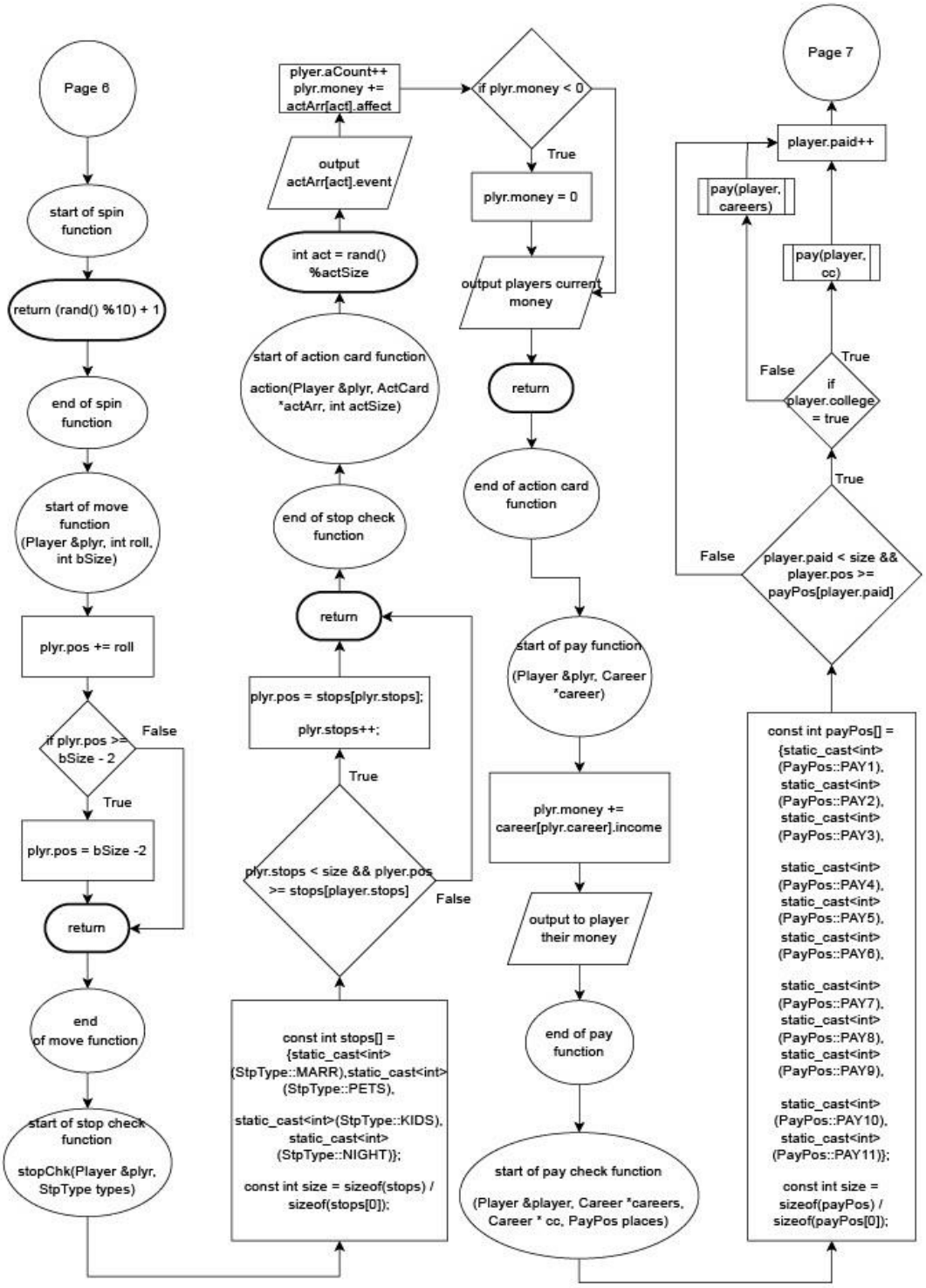

GameOfLIifeV2.cpp
/*
* File: GameOfLifeV2.cpp
* Author: Keanu Arao
* Created on May 12 2025 12:01 AM
* Purpose: Recreate the board game "Life" in C++ with an emphasis on classes
* Implement "Would you like to see your cards" function
*/
//Include statements
#include <iostream>
#include <string> // Strings
#include <ctime> // Time for rand
#include <cstdlib> // Srand to set seed
#include <fstream> // Binary Files
#include <limits> // For numeric limits
// Header File inclusion
#include "Player.h"
#include "Career.h"
#include "Action.h"
#include "House.h"
#include "GameStructs.h"
#include "Pet.h"
using namespace std;
// File Function Prototypes
void fRead(string, Action *, int);
void fRead(string, House *, int);
void fRead(string, Career *, int);
void fRead(string, Pet *, int);
void fRead(string, char *, int);
//Board Game Function Prototypes
int spin();
void pay(Player &);
void getMarr(Player &, Player* , int);
void getKids(Player &);
void stopChk(Player &, StpType);
void payChck(Player &, PayPos);
void action(Player &, Action *, int);
void buyH(Player &, House *, int);
void sellH(Player &);
void job(Player &, Career *, Career *, int);
void jobSlct(Player &, Career *, int);
void nightSc(Player &, Career *, Career *, int);
void getPets(Player &, Pet *, int &);
void retire(Player *, House *, int);
// Program Game Function Prototypes
void winner(Player *, int);
Player *pCreate(int &, Career *, Career *, int);
void results(Player *, int);
void seeCards(Player &);
// void record -> record winner to file
// Template Function Prototypes
template <typename T>
void dispArr(const T*, int);
/// Class Implementation
void Player::setName(std::string n) {
name = n;}
int Player::getMny() const {
return money;}
Page 2

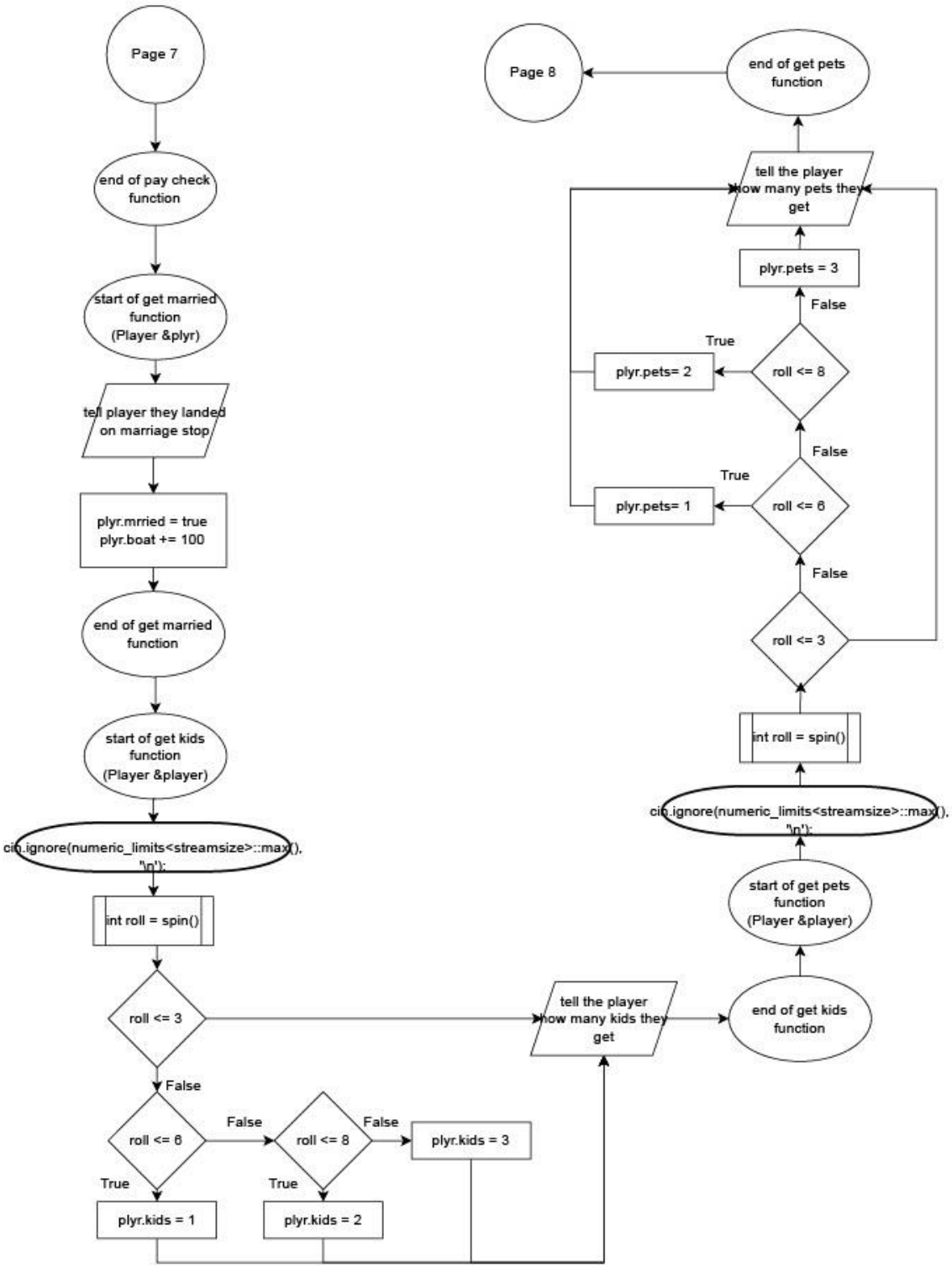

Page 2
Main
TRY
fRead("CollegeCareers.dat", cc, jSize);
fRead("Careers.dat", careers, jSize);
fRead("Houses.dat", houses, hSize);
fRead("board.dat", board, bSize);
fRead("Pets.dat", pets, petList);
fRead("ActionCards.dat", actions, aSize);
if const std::exception& e
True
std::cerr << "Exception: " << e.what() << std::endl;
return 1;
False
cout << "Enter the amount of players you want playing (Max of 4 players):";
while true
False
True
Prompt user to input into a char
if valid
False
True
break
call pCreate function to create players in player array
initialize eCount in do loop
output to tell user game is starting
declare roll and eCount (ints) outside of do-whileloop, initialize eCounts = 0 inside of do-while loop
eCount != pCount
False
Tell the users the game is over
retire(players, house, pCount) results(players, pCount)
delete[] players pointer array
return 0;
Page 3
True
eCount = 0
declare and initialize i
in for loop
i < pCount
False
i++
True
if player position == end of board
True
eCount++
continue;
False
tell user who's turn it is and have them press enter
roll = spin()
players[i].move(roll);
stopChk(players[i], stpType)
payChk(players[i], careers, cc, payPos)
call action(), buyH(), getMarr(), getPets(), nightSc() depending on player position

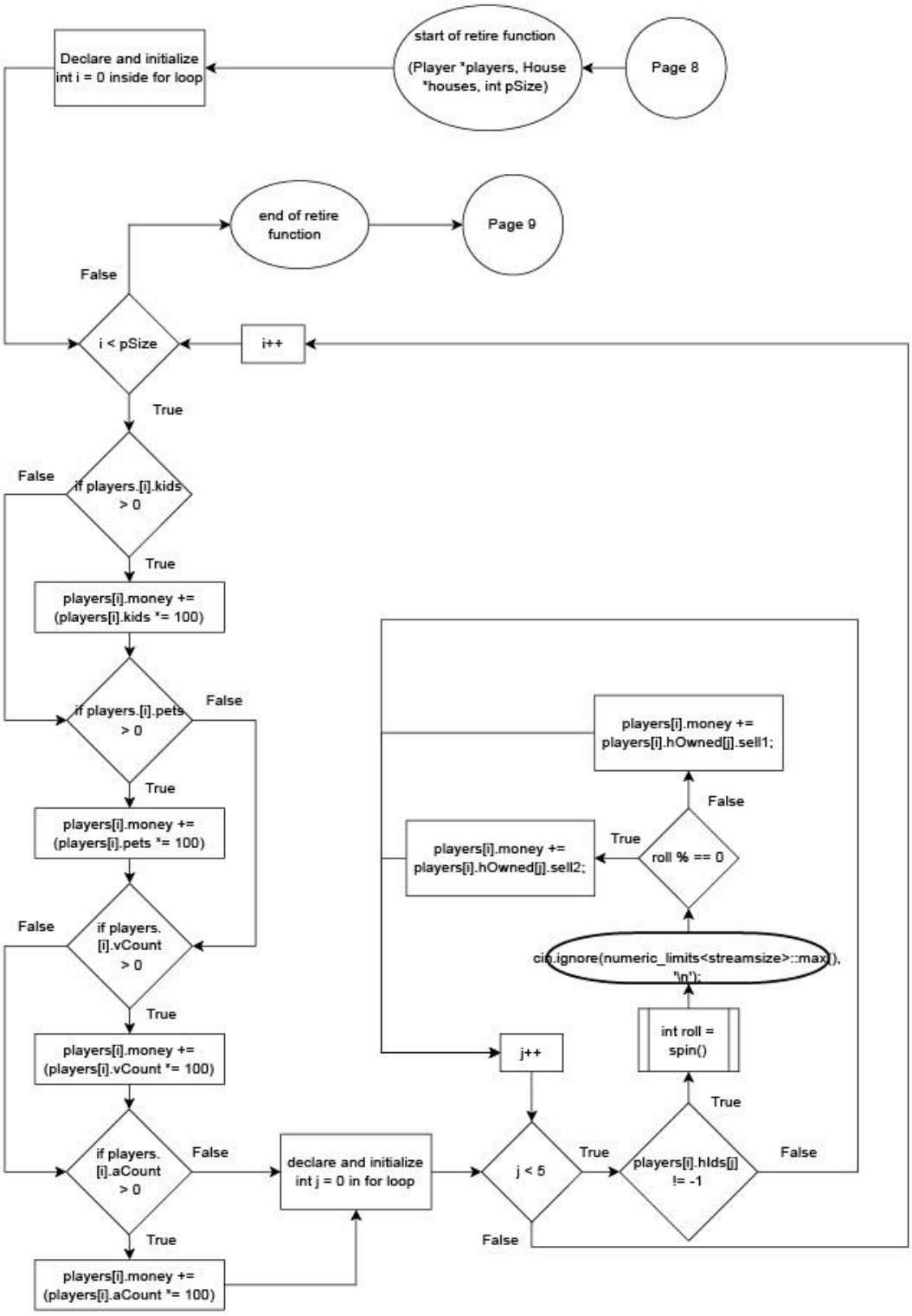

Page 3
start of house file reader
ifstream input(name, ios::binary)
if !input
True
return;
False
input.read(reinterpret_cast<char*> (arr), size * sizeof(House))
input.close()
end of house file reader
start of career file reader
ifstream input(name, ios::binary)
if !input
True
return;
False
input.read(reinterpret_cast<char*> (arr), size * sizeof(Career))
input.close()
end of career file reader
start of action card file reader
ifstream input(name, ios::binary)
if !input
True
return;
False
input.read(reinterpret_cast<char*> (arr), size * sizeof(actCard))
input.close()
end of action card file reader
start of board file reader
ifstream input(name, ios::binary)
if !input
True
return;
False
input.read(arr, size * sizeof(char))
input.close()
end of board file reader
Page 4

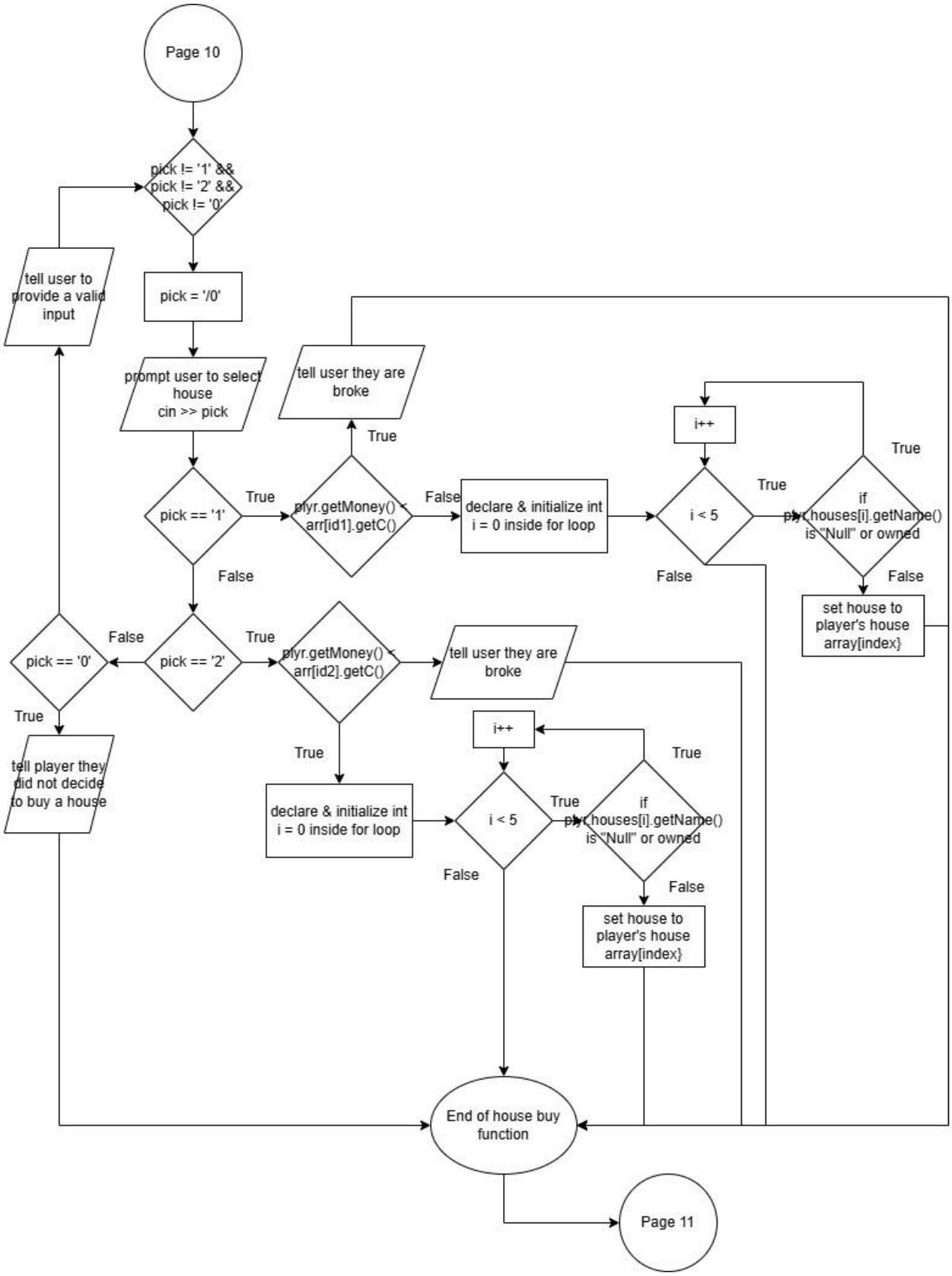

Page 4
start of *pCreate function (int &pCount, Career *cc, Career *careers, int jSize)
string name; Player *arr = new Player[pCount]
declare int i inside for loop
i < pCount
False
return arr
true
ask user to input a name put input into name and put it into arr[i].name
ask user if they want to go to college input into name again
if name = 'Y' or 'y'
True
arr[i].college = true
False
arr[i].college = false
i++
job(arr[i], careers, cc, jSize)
end of *pCreate function
start of route function (Player &player, char *board, Career *cc, Career *careers, House *houses, ActCard *actCard)
if board[player.pos] == 'A'
True
action(player, actCard, 40)
False
if board[player.pos] == 'P'
True
return
False
if board[player.pos] == 'V'
True
player.money += 100
False
if board[player.pos] == 'H'
True
house(player, houses, 20)
False
if board[player.pos] == 'M'
True
getMarr(player)
False
if board[player.pos] == 'C'
True
getPets(player)
False
if board[player.pos] == 'K'
True
getKids(player)
False
Page 5

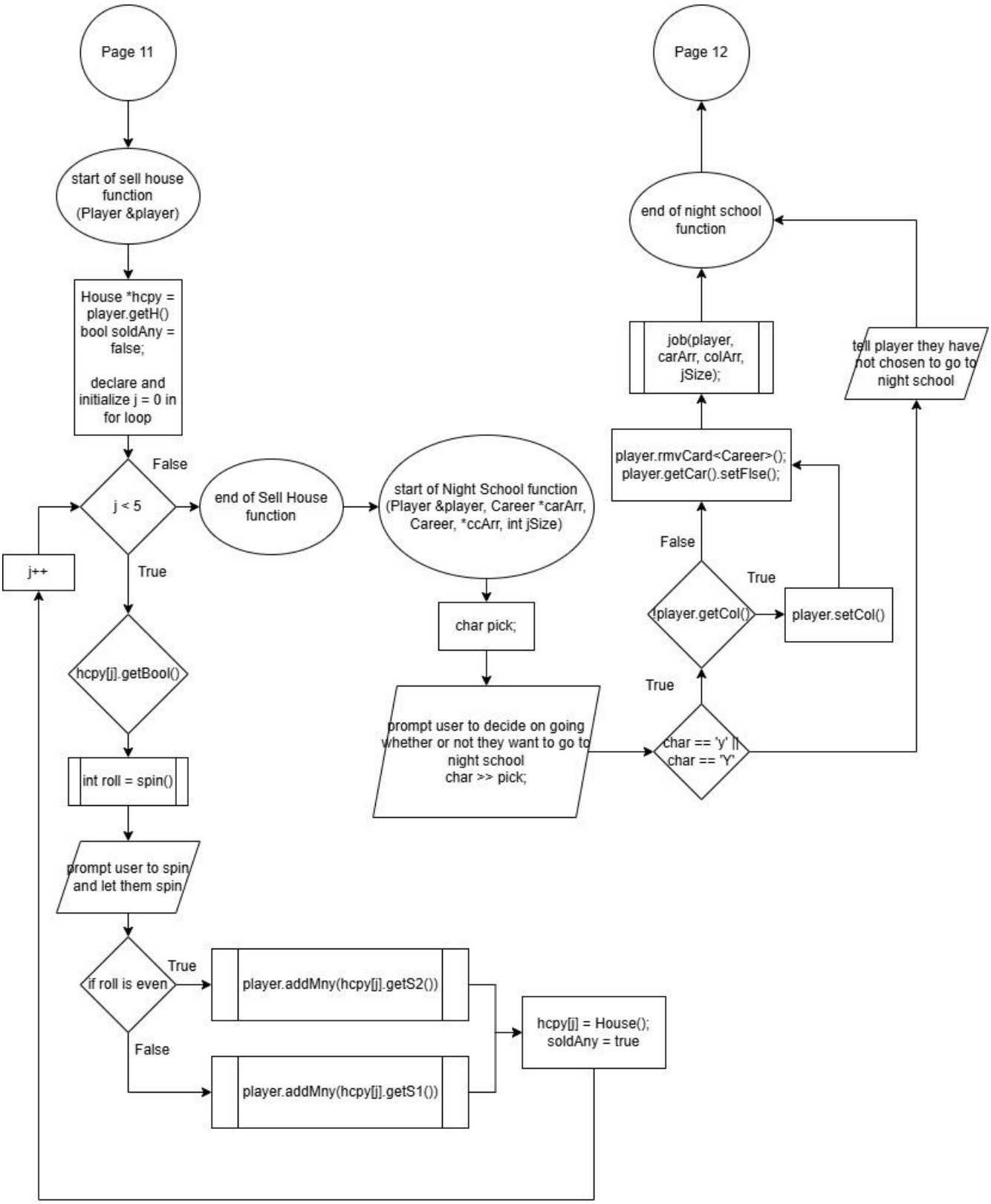

Page 5
char pick
Ask user if they'd like to go to night school
if board[player.pos] == 'N'
if user inputs "y" or "Y"
False
tell user that they did not decide to go to night school
True
job(arr[i], careers, cc, jSize)
if board[player.pos] == 'R'
True
tell the player they are retired
return;
end of route function
start of winner function (Player *players, int pSize)
declare outside for loop
int most = -1, mostId
bool tie = false
declare inside for loop
i = 0
i++
i < pSize
False
players[i].money > most
most = players[i].money
mostId = i
tie = false;
False
players[i].money == most
tie = true;
True
if tie == true
False
output that there was a tie
players[mostId].winner = true
output winner name
end of winner fuinction
start of results function (Player *players, int pSize)
char pick;
ask if the results would like to be seen
char == 'y' or 'Y'
False
True
declare and initialize int i inside for loop
i < pSize
True
False
display the player's results
end of winner fuinction
Page 6

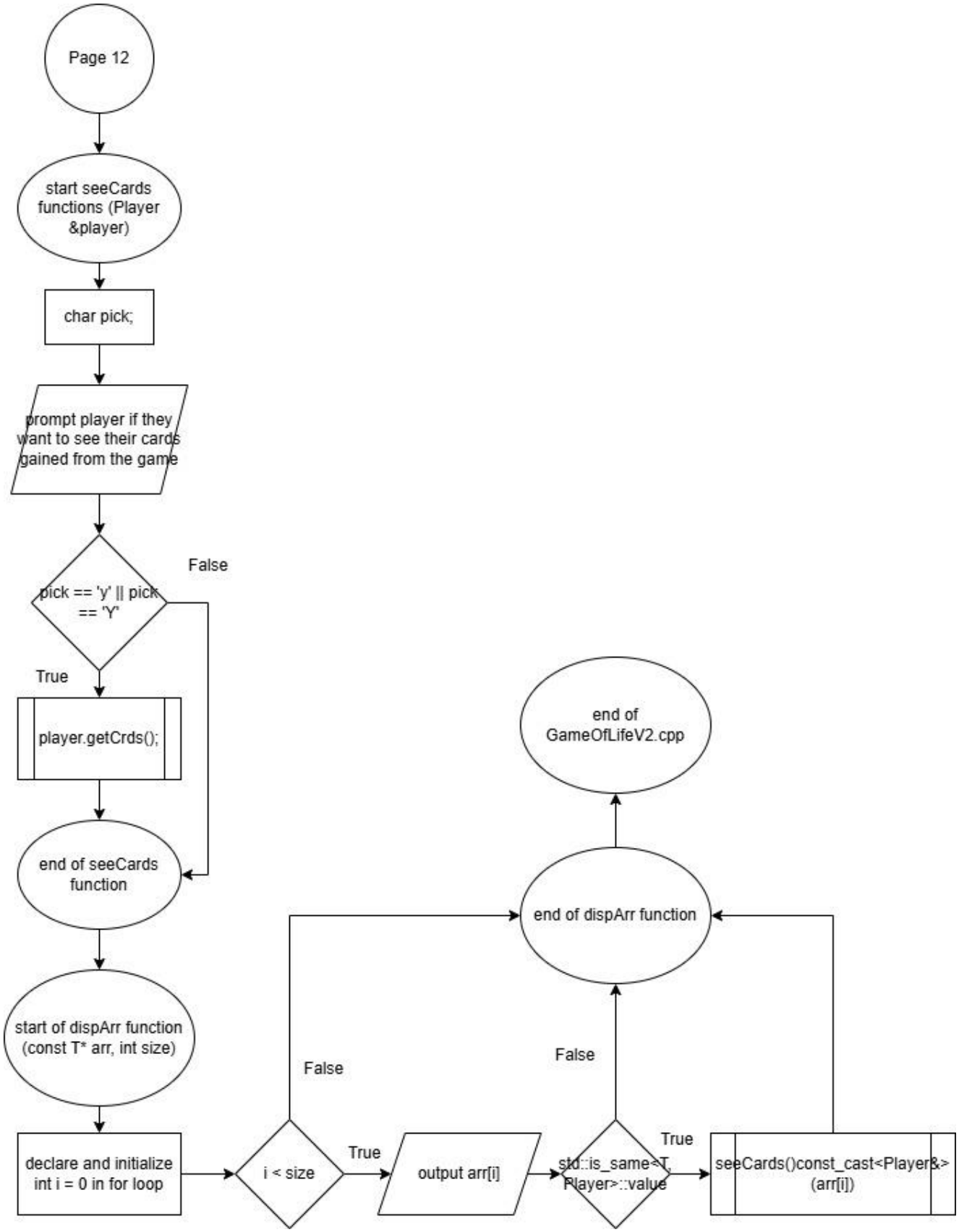

Page 6
start of spin function
return (rand() %10) + 1
end of spin function
start of move function (Player &plyr, int roll, int bSize)
plyr.pos += roll
if plyr.pos >= bSize - 2
False
True
plyr.pos = bSize -2
return
end of move function
start of stop check function
stopChk(Player &plyr, StpType types)
const int stops[] = {static_cast<int> (StpType::MARR),static_cast<int> (StpType::PETS),
static_cast<int>(StpType::KIDS), static_cast<int> (StpType::NIGHT)};
const int size = sizeof(stops) / sizeof(stops[0]);
plyr.stops < size && plyer.pos >= stops[player.stops]
True
False
plyr.pos = stops[plyr.stops];
plyr.stops++;
return
end of stop check function
start of action card function
action(Player &plyr, ActCard *actArr, int actSize)
int act = rand() %actSize
output actArr[act].event
plyer.aCount++ plyr.money += actArr[act].affect
if plyr.money < 0
True
plyr.money = 0
output players current money
return
end of action card function
start of pay function
(Player &plyr, Career *career)
plyr.money += career[plyr.career].income
output to player their money
end of pay function
start of pay check function
(Player &player, Career *careers, Career * cc, PayPos places)
const int payPos[] = {static_cast<int> (PayPos::PAY1), static_cast<int> (PayPos::PAY2), static_cast<int> (PayPos::PAY3),
static_cast<int> (PayPos::PAY4), static_cast<int> (PayPos::PAY5), static_cast<int> (PayPos::PAY6),
static_cast<int> (PayPos::PAY7), static_cast<int> (PayPos::PAY8), static_cast<int> (PayPos::PAY9),
static_cast<int> (PayPos::PAY10), static_cast<int> (PayPos::PAY11)};
const int size = sizeof(payPos) / sizeof(payPos[0]);
player.paid < size && player.pos >= payPos[player.paid]
False
True
if player.college = true
False
True
pay(player, careers)
pay(player, cc)
player.paid++
Page 7


Page 7
end of pay check function
start of get married function (Player &plyr)
tell player they landed on marriage stop
plyr.mrried = true
plyr.boat += 100
end of get married function
start of get kids function (Player &player)
cin.ignore(numeric_limits<streamsize>::max(), '\n');
int roll = spin()
roll <= 3
False
roll <= 6
True
plyr.kids = 1
False
roll <= 8
True
plyr.kids = 2
False
plyr.kids = 3
tell the player how many kids they get
end of get kids function
start of get pets function (Player &player)
cin.ignore(numeric_limits<streamsize>::max(), '\n');
int roll = spin()
roll <= 3
False
roll <= 6
True
plyr.pets= 1
False
roll <= 8
True
plyr.pets= 2
False
plyr.pets = 3
tell the player how many pets they get
end of get pets function
Page 8


start of retire function
(Player *players, House *houses, int pSize)
Page 8
Declare and initialize int i = 0 inside for loop
end of retire function
Page 9
False
i < pSize
i++
True
False
if players.[i].kids > 0
True
players[i].money += (players[i].kids *= 100)
if players.[i].pets > 0
False
True
players[i].money += (players[i].pets *= 100)
False
if players. [i].vCount > 0
True
players[i].money += (players[i].vCount *= 100)
if players. [i].aCount > 0
False
True
players[i].money += (players[i].aCount *= 100)
declare and initialize int j = 0 in for loop
j < 5
True
False
players[i].hIds[j] != -1
True
False
int roll = spin()
cin.ignore(numeric_limits<streamsize>::max(), '\n');
roll % == 0
True
players[i].money += players[i].hOwned[j].sell2;
False
players[i].money += players[i].hOwned[j].sell1;
j++

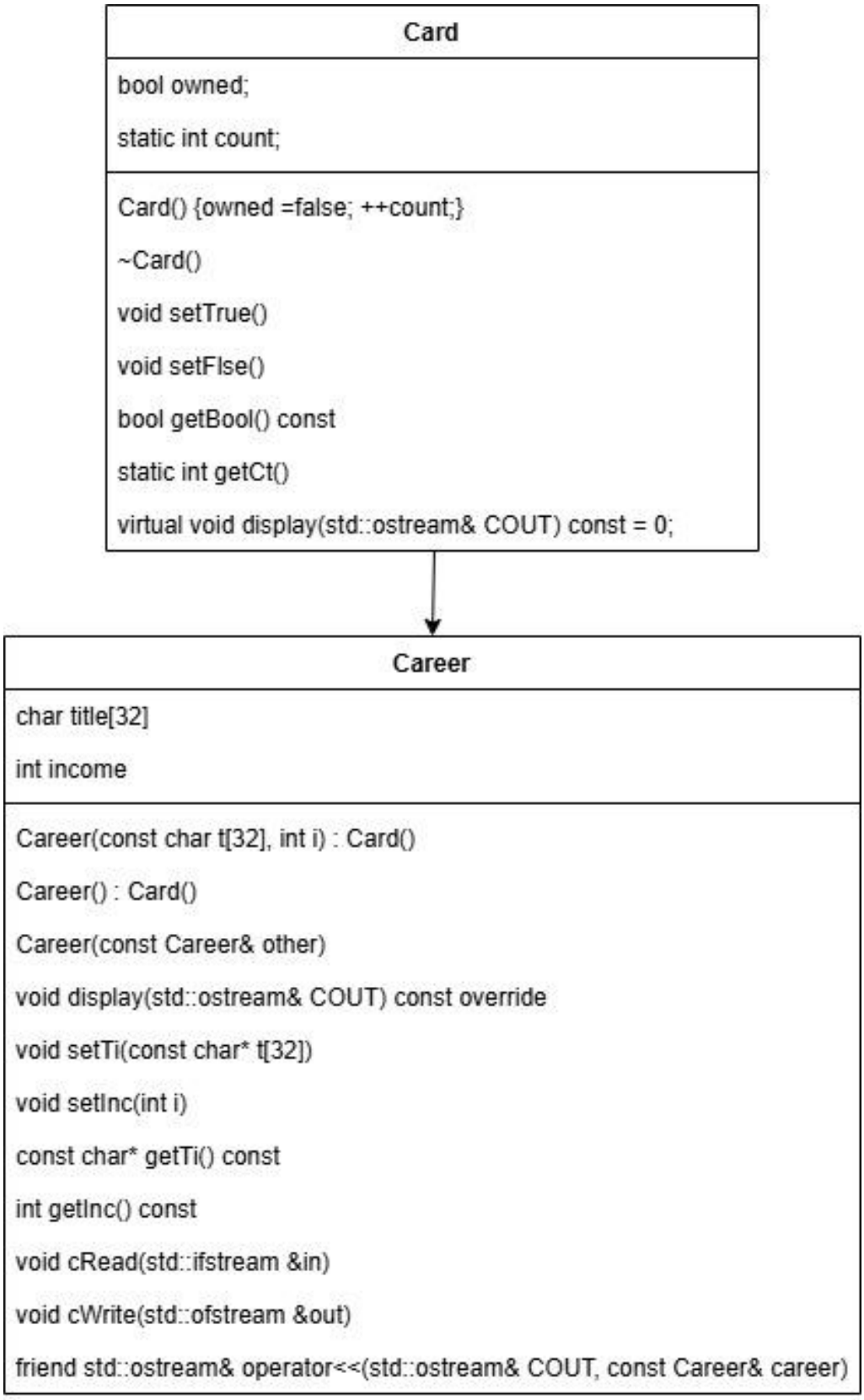

Page 10
pick != '1' &&
pick != '2' &&
pick != '0'
tell user to provide a valid input
pick = '/0'
prompt user to select house
cin >> pick
tell user they are broke
True
pick == '1'
True
plyr.getMoney() <
arr[id1].getC()
False
declare & initialize int
i = 0 inside for loop
i++
i < 5
True
False
True
if
plyr.houses[i].getName()
is "Null" or owned
False
set house to
player's house
array[index}
False
pick == '0'
False
pick == '2'
True
plyr.getMoney() <
arr[id2].getC()
tell user they are broke
True
tell player they did not decide to buy a house
True
declare & initialize int
i = 0 inside for loop
i++
i < 5
True
False
True
if
plyr.houses[i].getName()
is "Null" or owned
False
set house to
player's house
array[index}
End of house buy function
Page 11

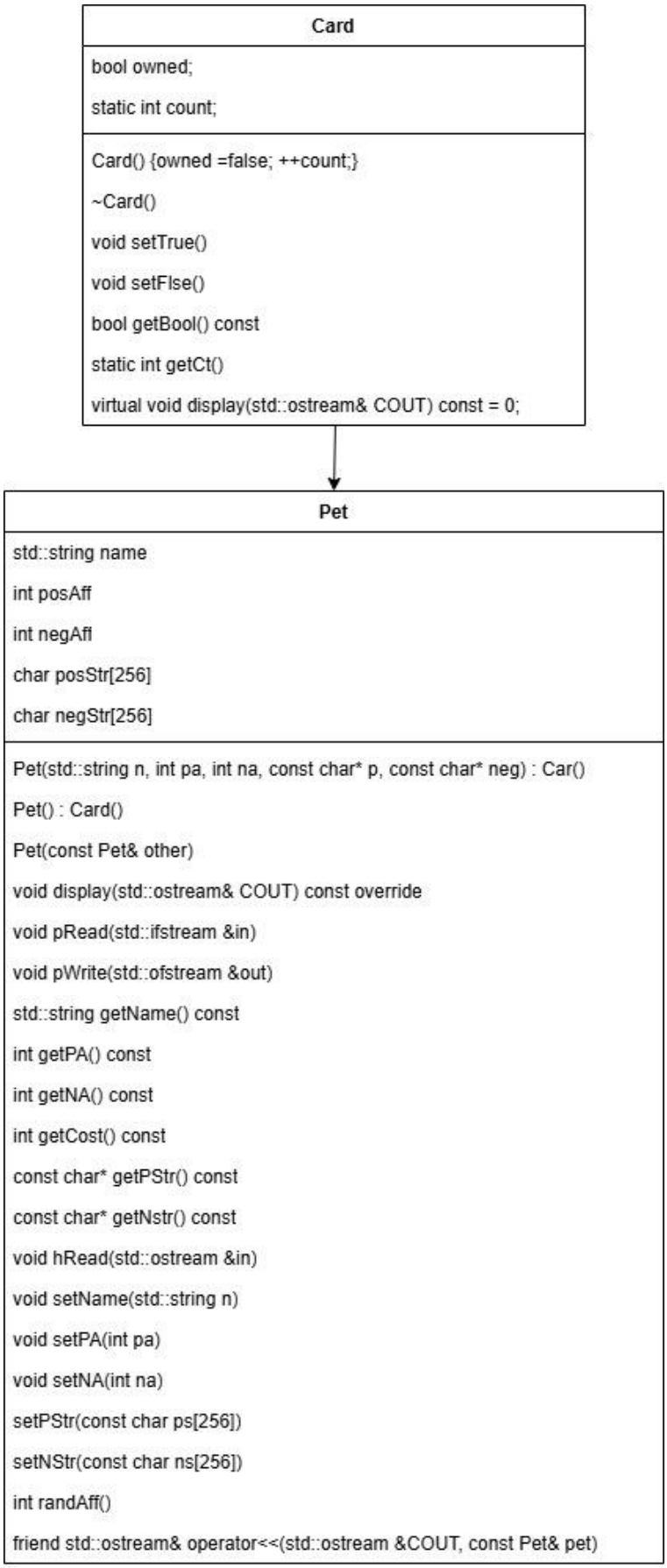

Page 11
start of sell house function (Player &player)
House *hcpy = player.getH() bool soldAny = false;
declare and initialize j = 0 in for loop
j < 5
False
True
j++
hcpy[j].getBool()
int roll = spin()
prompt user to spin and let them spin
if roll is even
True
False
player.addMny(hcpy[j].getS2())
player.addMny(hcpy[j].getS1())
hcpy[j] = House(); soldAny = true
end of Sell House function
start of Night School function (Player &player, Career *carArr, Career, *ccArr, int jSize)
char pick;
prompt user to decide on going whether or not they want to go to night school
char >> pick;
char == 'y' || char == 'Y'
True
!player.getCol()
False
True
player.setCol()
player.rmvCard<Career>(); player.getCar().setFlse();
job(player, carArr, colArr, jSize);
tell player they have not chosen to go to night school
end of night school function
Page 12

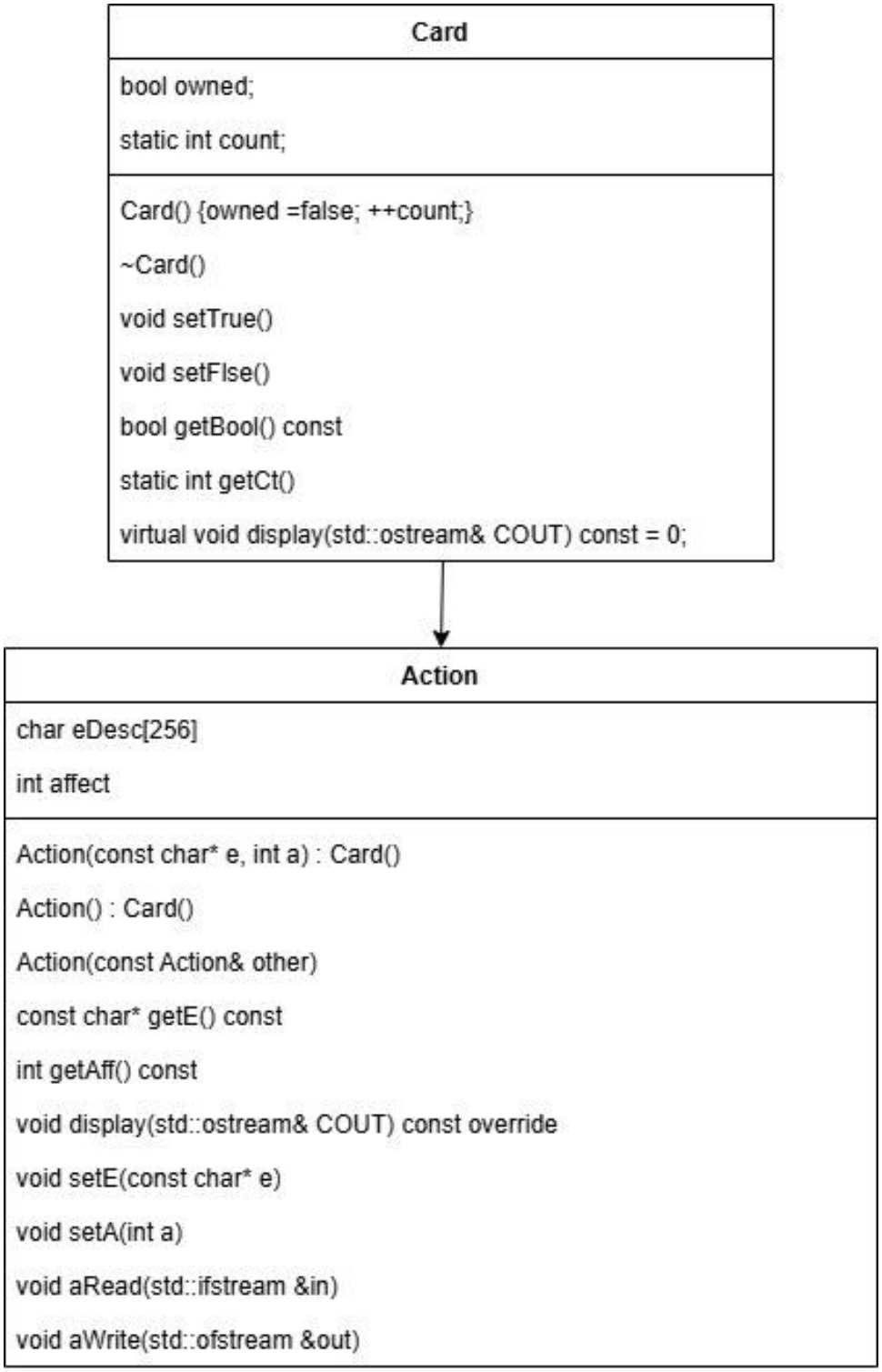

Page 12
start seeCards functions (Player &player)
char pick;
prompt player if they want to see their cards gained from the game
pick == 'y' || pick == 'Y'
False
True
player.getCrds();
end of seeCards function
start of dispArr function (const T* arr, int size)
declare and initialize int i = 0 in for loop
i < size
True
False
output arr[i]
std::is_same<T, Player>::value
True
False
seeCards()const_cast<Player&> (arr[i])
end of dispArr function
end of GameOfLifeV2.cpp

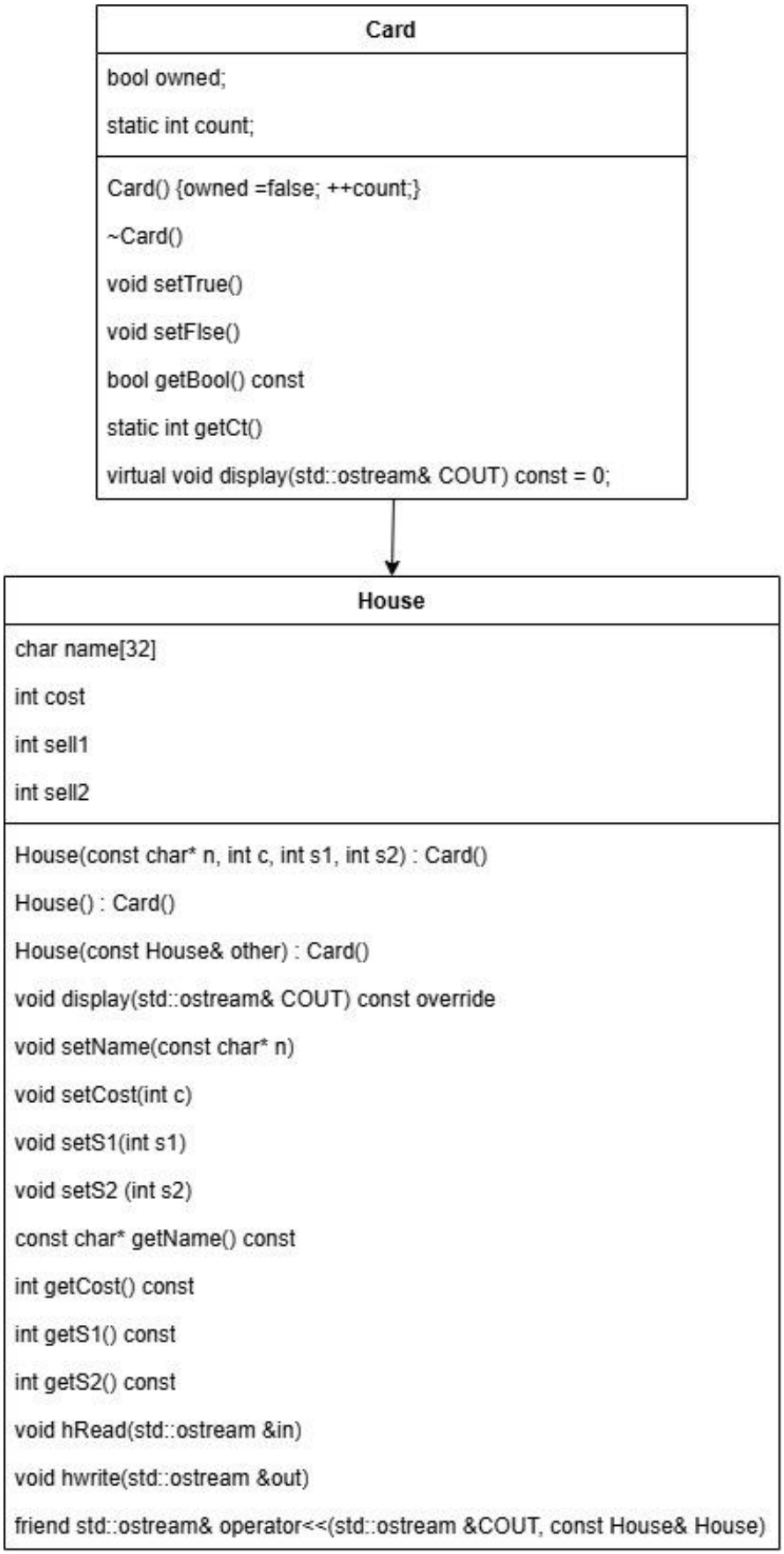
UML

Player
std::string name static int turns; int money=200, pos = 0, aCount = 0, stops = 0, paid = 0, kids = 0, vCount = 0 bool college = false, mrried = false, winnner = false Career career House houses[5] Pet pet
Player(std::string n, bool c) Player() Player(const Player& other) void setName(std::string n) void setCar(Career c) void addMny(int m) void move(int r, int max) void setPots(int p) int addV() int addA() void setCol() void setMrr() void setWin() void setKids(int k) void addStop() void addPaid() void setMny(int m) void setH(const House *h) void setPet(Pet p) void addCard(std::unique_ptr<Card> card) voidRmv() std::string getName() const int getMny() const int getPos() const int getAC() const int getVC() const int getKids() const int getStps() const int paidNum() const bool getCol() const bool getMrr() const bool getWin() const Career getCar() const House *getH() Pet getPet() int getTurns() void getCrds() const friend std::ostream& operator<<(std::ostream& COUT, const Player& player) friend Player operator+(const Player& a, const Player& b) friend Player operator==(const Player& a, const Player& b) friend Player operator!=(const Player& a, const Player& b) Player& operator=(const Player& other)


Card
bool owned;
static int count;
Card() {owned =false; ++count;}
~Card()
void setTrue()
void setFlse()
bool getBool() const
static int getCt()
virtual void display(std::ostream& COUT) const = 0;
Career
char title[32]
int income
Career(const char t[32], int i) : Card()
Career() : Card()
Career(const Career& other)
void display(std::ostream& COUT) const override
void setTi(const char* t[32])
void setInc(int i)
const char* getTi() const
int getInc() const
void cRead(std::ifstream &in)
void cWrite(std::ofstream &out)
friend std::ostream& operator<<(std::ostream& COUT, const Career& career)


Card
bool owned;
static int count;
Card() {owned =false; ++count;}
~Card()
void setTrue()
void setFlse()
bool getBool() const
static int getCt()
virtual void display(std::ostream& COUT) const = 0;
Pet
std::string name
int posAff
int negAfl
char posStr[256]
char negStr[256]
Pet(std::string n, int pa, int na, const char* p, const char* neg) : Car()
Pet() : Card()
Pet(const Pet& other)
void display(std::ostream& COUT) const override
void pRead(std::ifstream &in)
void pWrite(std::ofstream &out)
std::string getName() const
int getPA() const
int getNA() const
int getCost() const
const char* getPStr() const
const char* getNstr() const
void hRead(std::ostream &in)
void setName(std::string n)
void setPA(int pa)
void setNA(int na)
setPStr(const char ps[256])
setNStr(const char ns[256])
int randAff()
friend std::ostream& operator<<(std::ostream &COUT, const Pet& pet)


Card
bool owned;
static int count;
Card() {owned =false; ++count;}
~Card()
void setTrue()
void setFlse()
bool getBool() const
static int getCt()
virtual void display(std::ostream& COUT) const = 0;
Action
char eDesc[256]
int affect
Action(const char* e, int a) : Card()
Action() : Card()
Action(const Action& other)
const char* getE() const
int getAff() const
void display(std::ostream& COUT) const override
void setE(const char* e)
void setA(int a)
void aRead(std::ifstream &in)
void aWrite(std::ofstream &out)


Card
bool owned;
static int count;
Card() {owned =false; ++count;}
~Card()
void setTrue()
void setFlse()
bool getBool() const
static int getCt()
virtual void display(std::ostream& COUT) const = 0;
House
char name[32]
int cost
int sell1
int sell2
House(const char* n, int c, int s1, int s2) : Card()
House() : Card()
House(const House& other) : Card()
void display(std::ostream& COUT) const override
void setName(const char* n)
void setCost(int c)
void setS1(int s1)
void setS2 (int s2)
const char* getName() const
int getCost() const
int getS1() const
int getS2() const
void hRead(std::ostream &in)
void hwrite(std::ostream &out)
friend std::ostream& operator<<(std::ostream &COUT, const House& House)

Variables

The major variables and arrays that are used throughout the main program.

TYPE	NAME	LOCATION (LINE)
CONST INT	jList	54
CONST INT	aList	54
CONST INT	bSize	54
INT	pCount	55
CHAR	board[bSize]	56
Career (CLASS)	careers[jList]	57
Career (CLASS)	cc[jList]	58
House (CLASS)	houses[jList]	59
Actions (CLASS)	actCard[aList]	60
StpType (ENUM STRUCT)	stpType	61
PayPos (ENUM STRUCT)	payPos	62

Concepts

Here are the concepts from chapters 9-12 of the Gaddis textbook that were learned and used for this project.

Chapter	Section	Topic	Where Line #"s	Pts
13		Classes		
	1 to 3	Instance of a Class	"GameOfLifeV2.cpp": 74-81	4
	4	Private Data Members	“Player.h”: 14-24 “Pet.h”: 14-17 “Action.h”: 9-11 “Career.h”: 8-10 “House.h”: 7-9	4

	5	Specification vs. Implementation	Specification: “Player.h”: 51, 97 Implementation: “GameOfLifeV2.cpp": 61-66	4
	6	Inline	“Player.h”: 51-116 “Pet.h”: 80-84, 96-100 “Action.h”: 31-32, 41-44 “Career.h”: 29-34 “House.h”: 45-57 “Card.h”: 16-20	4
	7, 8, 10	Constructors	“Player.h”: 26-27 “Pet.h”: 19-40 “Action.h”: 13-22 “Career.h”: 13-14 “House.h”: 12-26 “Card.h”: 13	4
	9	Destructors	“Card.h”: 14	4
	12	Arrays of Objects	“GameOfLifeV2.cpp": 75-80 “GameOfLifeV2_SecondaryFile.cpp”: 50, 59, 68, 77, 86 “Player.h”: 21	4
	16	UML	In “Flowcharts/Psuedo Code/UML”	4
14		More about Classes		
	1	Static	“Player.h”: 16 “Card.h”: 11, 20	5
	2	Friends	“Player.h”: 122, 136, 149, 157 “Pet.h”: 125 “Career.h”: 54 “House.h”: 81	2
	4	Copy Constructors	“Player.h”: 30 “Pet.h”: 42 “Action.h”: 24 “Career.h”: 17 “House.h”: 29	5
	5	Operator Overloading	“Player.h”: 122, 136, 149, 157 “Pet.h”: 125 “Career.h”: 54 “House.h”: 81	8
	7	Aggregation	“Player.h”: 20-22	6

15		Inheritance		
	1	Protected members	“Card.h”: 9-11	6
	2 to 5	Base Class to Derived	Base: “Card.h”: Derived: “Pet.h”: 13 “Action.h”: 8 “Career.h”: 7 “House.h”: 6	6
	6	Polymorphic associations	“Pet.h”: 87-92 “Action.h”: 35-38 “Career.h: 23-26 “House.h”: 37-42	6
	7	Abstract Classes	“Card.h”: 23	6
16		Advanced Classes		
	1	Exceptions	“GameOfLifeV2.cpp”: 83-93 “GameOfLifeV2_SecondaryFile.cpp”: 52-57, 61-66, 70-75, 79-84,103-107, 119, 128, 139, 150, 161	6
	2 to 4	Templates	“GameOfLifeV2.cpp”: 56-57,325-333 “Player.h”: 83-93	6
	5	STL	“Player.h”: 24, 79, 83-93, 112-116	6
		Sum		100

References/Sources

Listed in this section are the sources used for certain concepts/help needed to continue with the project:

https://stackoverflow.com/questions/903221/press-enter-to-continue

https://www.youtube.com/watch?v=BnMnozsSPmw&ab_channel=CodeBeauty

https://stackoverflow.com/questions/4273375/why-is-the-c-stdexceptionwhat-member-const

Program

THIS IS THE MAIN FILE USED FOR THE PROGRAM (OTHER FILES ARE NOT INCLUDED)

```
/*
 * File:   GameOfLifeV2.cpp
 * Author: Keanu Arao
 * Created on May 12 2025 12:01 AM
 * Purpose:  Recreate the board game "Life" in C++ with an emphasis on classes
 *          Implement "Would you like to see your cards" function
 */

//Include statements
```

```
#include <iostream>
#include <string> // Strings
#include <ctime> // Time for rand
#include <cstdlib> // Srand to set seed
#include <fstream> // Binary Files
#include <limits> // For numeric limits
// Header File inclusion
#include "Player.h"
#include "Career.h"
#include "Action.h"
#include "House.h"
#include "GameStructs.h"
#include "Pet.h"

using namespace std;

// File Function Prototypes
void fRead(string, Action *, int);
void fRead(string, House *, int);
void fRead(string, Career *, int);
void fRead(string, Pet *, int);
void fRead(string, char *, int);

//Board Game Function Prototypes
int spin();
void pay(Player &);
void getMarr(Player &, Player* , int);
void getKids(Player &);
void stopChk(Player &, StpType);
void payChck(Player &, PayPos);
void action(Player &, Action *, int);
void buyH(Player &, House *, int);
void sellH(Player &);
void job(Player &, Career *, Career *, int);
void jobSlct(Player &, Career *, int);
void nightSc(Player &, Career *, Career *, int);
void getPets(Player &, Pet *, int &);
void retire(Player *, House *, int);

// Program Game Function Prototypes
void winner(Player *, int);
Player *pCreate(int &, Career *, Career *, int);
void results(Player *, int);
```

```
void seeCards(Player &);
// void record -> record winner to file
// Template Function Prototypes
template <typename T>
void dispArr(const T*, int);

// Main Program
int main(){
    /// Setting the Random seed for rand functions
    srand(static_cast<unsigned int>(time(0)));

    int jSize = 20, hSize = 20, bSize = 100, pCount, aSize = 70, petList = 10;
    Career cc[jSize], careers[jSize];
    Action actions[aSize];
    House houses[hSize];
    Pet pets[petList];
    StpType stpType;
    PayPos payPos;
    char board[bSize];

    try {
        fRead("CollegeCareers.dat", cc, jSize);
        fRead("Careers.dat", careers, jSize);
        fRead("Houses.dat", houses, hSize);
        fRead("board.dat", board, bSize);
        fRead("Pets.dat", pets, petList);
        fRead("ActionCards.dat", actions, aSize);
    } catch (const std::exception& e) {
        std::cerr << "Exception: " << e.what() << std::endl;
        return 1;
    }

    cout << "Enter the amount of players you want playing (Max of 4 players):";

    while (true) {
        cin >> pCount;
        if (pCount > 4 || pCount <= 0) {
            cin.clear(); // Clear the error flag
            cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Discard
invalid input
            cout << "Invalid input. Enter a number (1-4)"<< endl;
            cout << ">";
        } else {
```

```
            break; // Exit loop if input is valid
        }
    }

    cin.ignore();
    Player *players = new Player[pCount];
    players = pCreate(pCount, cc, careers, jSize); // Create players (name,
money, etc...)

    cout << "========================================" << endl;
    cout << "Game starting..." << endl;
    int roll, eCount; // counter for end game check
    do {
        eCount = 0;
        for (int i = 0; i < pCount; i++){
            if (players[i].getPos() == bSize - 2){
                eCount++;
                continue; // Skip to next player
            }
            cout << "========================================" << endl;
            cout << players[i].getName() << "'s turn!" << endl;
            cout << "Tap enter to spin: ";
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            roll = spin();
            cout << "You rolled a(n) " << roll << endl;
            players[i].move(roll, bSize - 2); // Move Player
            stopChk(players[i], stpType); // Check to see if player passes/lands
on stop
            payChck(players[i], payPos); // Call pay function (if they pass a pay
position)
            cout << board[players[i].getPos()] << endl;

            // -----ROUTE FUNCTION RE-IMPLEMENTED INTO MAIN GAME LOOP-----
            if (board[players[i].getPos()] == 'A'){ // Action space
                action(players[i], actions, 40); // Call action function
            }
            else if (board[players[i].getPos()] == 'P') {
                continue;
            }
            else if (board[players[i].getPos()] == 'V') {
                cout << "You have landed on a vacation spot! Get a 100K bonus!"
<< endl;
                players[i].addV();
```

```
            }
            else if (board[players[i].getPos()] == 'H'){ // House space
                buyH(players[i], houses, hSize); // Call house function
            }
            else if (board[players[i].getPos()] == 'M'){ // Marriage space
                getMarr(players[i], players, pCount); // Call marriage function
            }
            else if (board[players[i].getPos()] == 'C') {
                getPets(players[i], pets, petList);
            }
            else if (board[players[i].getPos()] == 'K'){ // Kids space
                getKids(players[i]); // Call kids function
            }
            else if (board[players[i].getPos()] == 'N') { // Night School
                nightSc(players[i],careers, cc, jSize);
            }
            else if (board[players[i].getPos()] == 'R'){
                cout << "You are retired." << endl;
                continue;
            }
            else {cout << "Error" << endl;}

        } // End of for loop
    }while(eCount != pCount); // If all players are at the end of the board, end
game

    retire(players, houses, pCount);
    results(players, pCount);

    delete[] players;
    return 0;
}

//=================================================================------FILE
FUNCTIONS------
=================================================================================
====

void fRead(string name, char *arr, int size){ // Board reader
    ifstream input(name, ios::binary);
    if (!input){
        throw std::runtime_error("Error opening file: " + name);
    }
```

```
    input.read(arr, size * sizeof(char));
    input.close();
}

void fRead(string name, Action *arr, int size){ // Action Card Reader
    ifstream input(name, ios::binary);
    if (!input){
        throw std::runtime_error("Error opening file: " + name);
    }
    for (int i = 0; i < size; i++) {
        arr[i].aRead(input);
    }
    input.close();
}

void fRead(string name, Career *arr, int size){ // Career Card Reader
    ifstream input(name, ios::binary);
    if (!input){
        throw std::runtime_error("Error opening file: " + name);
    }
    for (int i = 0; i < size; i++) {
        arr[i].cRead(input);
    }
    input.close();
}

void fRead(string name, House *arr, int size){  // House Card Reader
    ifstream input(name, ios::binary);
    if (!input){
       throw std::runtime_error("Error opening file: " + name);
    }
    for (int i = 0; i < size; i++) {
        arr[i].hRead(input);
    }
    input.close();
}

void fRead(string name, Pet *arr, int size){  // Pet Card Reader
    ifstream input(name, ios::binary);
    if (!input){
        throw std::runtime_error("Error opening file: " + name);
    }
    for (int i = 0; i < size; i++) {
```

```
        arr[i].pRead(input);
    }
    input.close();
}

//=================================================================------PROGAM
GAME FUNCTIONS------
=================================================================================
====

/**
 * @brief   Create a player array and assigning them names and careers
 *
 * @param pCount    Player count
 * @param cc        College Career Array
 * @param careers   Regular Career array
 * @param jSize     Size of career array(s)
 * @return Player* Return Array of Players
 */
Player *pCreate(int &pCount, Career *cc, Career *careers, int jSize){
    string name;
    char in;
    Player *arr = new Player[pCount];
    for (int i = 0; i < pCount; i++){
        cout << "Enter player " << i + 1 << "'s name: ";
        getline(cin, name);
        arr[i].setName(name);
        cout << "Would you like to go to college? (Y/N): "; cin >> in;
        if (in == 'Y' || in == 'y'){
            arr[i].setCol();
        }
        job(arr[i], careers, cc, jSize);
        cin.ignore();
    }
    return arr;
}
/**
 * @brief Determines the winner of the game by how much money they have
 *
 * @param players Player array
 * @param pSize size of Player array
 */
void winner(Player *players, int pSize) {
    int most = -1, mostId;
```

```
    bool tie = false;
    for (int i = 0; i < pSize; i++) {
        if (players[i].getMny() > most) {
            most = players[i].getMny();
            mostId = i;
            tie = false;
        }
        else if (players[i].getMny() == most) {
            tie = true;
        }
    }
    if (tie) {
        cout << "There was a tie!" << endl;
    }
    else {
        players[mostId].setWin();
        cout << "The Winner is " << players[mostId].getName() << "!" << endl;
    }
}
/**
 * @brief prints out the results of the game
 *
 * @param players Player array
 * @param pSize Player array size
 */
void results(Player *players, int pSize){
    char pick;
    cout << "=========================================" << endl;
    cout << "Would you like to see the results? (Y/N): "; cin >> pick;
    if (pick == 'Y' || pick == 'y'){
        cout << "-----------------------------------------" << endl;
        dispArr(players, pSize);

        cout << "-----------------------------------------" << endl;
        winner(players, pSize);
        cout << "-----------------------------------------" << endl;
        cout << "Amount of turns taken in the game: " << players[0].getTurns() <<
endl;
        Player temp = players[0];
        for (int i = 1; i < pSize; i++) {
            temp = temp + players[i];
        }
        cout << "Total Money gained in game: " << temp.getMny() << endl;
        cout << "Total Action Cards gained in Game: " << temp.getAC() << endl;
```

```
        cout << "Total Vacation Cards gained in Game: " << temp.getVC() << endl;
        cout << "Total Children acquired (or made you could say) in game: " <<
temp.getKids() << endl;
        cout << "==========================================" << endl;
    }
    else {cout << "You have chosen to not see the results." << endl;}
    cin.ignore();
}

/**
 * @brief Template function for printing out an array of cards
 *
 * @tparam T -> the template Type placeholder
 * @param arr The array of object type
 * @param size The array size
 */
template <typename T>
void dispArr(const T* arr, int size){
    for (int i = 0; i < size; i++) {
        cout << arr[i] << endl;
        if constexpr (std::is_same<T, Player>::value) { // Check if object type
of Player
            seeCards(const_cast<Player&>(arr[i]));
        }
    }
}

/**
 * @brief Displays all the cards a player got during the game
 *
 * @param player The player to show cards to
 */
void seeCards(Player &player) {
    cin.ignore();
    char pick;
    cout << "Would you like to see the cards you recieved during the game? (y/n)"
<< endl;
    cin >> pick;
    if (pick == 'y' || pick == 'Y') {
        player.getCrds();
    }
    else{
        cout << "You have decided to not see the cards you recieved." << endl;
    }
```

```
    cin.ignore();
}
//==================================================================------CORE/BOARD
GAME FUNCTIONS------
==================================================================================
====

int spin() {
    return (rand() % 10) + 1;
}

/// @brief Pays the player their income
/// @param player The Player
void pay(Player &player){
    cout << "You have passed a pay position!" << endl;
    player.addMny(player.getCar().getInc());
    cout << "You now have " << player.getMny() << "K" << endl;
}

/// @brief Makes sure to place the player in the correct spot when hypothetically
passing stop indexes
/// @param plyr the player
/// @param types enum struct to hold array of stops (ints)
void stopChk(Player &plyr, StpType types) { // Check if player has landed on a
stop (or passed it) and if they have, place them on the stop
    const int stops[] =
{static_cast<int>(StpType::MARR),static_cast<int>(StpType::PETS),
                        static_cast<int>(StpType::KIDS),
static_cast<int>(StpType::NIGHT)}; // Stops array (for now)
    const int size = sizeof(stops) / sizeof(stops[0]);

    if (plyr.getStps() < size && plyr.getPos() >= stops[plyr.getStps()]){
        plyr.setPos(stops[plyr.getStps()]); // Place them on the stop
        plyr.addStop();
    }
}

/// @brief Makes sure to not pay the player every single move by keeping count of
how many pay positions the player has passed
/// @param player the player
/// @param places enum struct to hold pay position (ints)
void payChck(Player &player, PayPos places){
```

```
    const int payPos[] = {static_cast<int>(PayPos::PAY1),
static_cast<int>(PayPos::PAY2), static_cast<int>(PayPos::PAY3),
                          static_cast<int>(PayPos::PAY4),
static_cast<int>(PayPos::PAY5), static_cast<int>(PayPos::PAY6),
                          static_cast<int>(PayPos::PAY7),
static_cast<int>(PayPos::PAY8), static_cast<int>(PayPos::PAY9),
                          static_cast<int>(PayPos::PAY10),
static_cast<int>(PayPos::PAY11)};
    const int size = sizeof(payPos) / sizeof(payPos[0]);
    if (player.paidNum() < size && player.getPos() >= payPos[player.paidNum()]) {
        pay(player);
        player.addPaid();
    }
}

/// @brief When a player lands on the marry stop, all other players pay that
player 100K
/// @param plyr The Player getting paid
/// @param players The Player Array
/// @param pCount  The Player Array size
void getMarr(Player &plyr, Player* players, int pCount){
    cout << "You have landed on the marriage STOP!" << endl;
    plyr.setMrr();
    for (int i = 0; i < pCount; i++) {
         if (players[i] != plyr) {
            players[i].addMny(-100);
            if (players[i].getMny() < 0) {
                players[i].setMny(0);
            }
            plyr.addMny(100);
         }
    }
    cout << "You are now married! You receive 100K from all the other players!"
<< endl;
}

/// @brief When the player lands on the kids stop, they spin to get a certain
amount of kids
/// @param plyr The player getting these kids
void getKids(Player &plyr){
    cout << "You have landed on the kids STOP! Spin (Enter) for kids!" << endl;
    cout << ">";
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    int roll = spin();
```

```
    cout << "You rolled a(n) " << roll << endl;
    if (roll <= 3) {
        cout << "You get no kids!" << endl;
    } else if (roll <= 6) {
        cout << "You have 1 kid!" << endl;
        plyr.setKids(1);
    } else if (roll <= 8) {
        cout << "You have twins!" << endl;
        plyr.setKids(2);
    } else {
        cout << "You have triplets!" << endl;
        plyr.setKids(3);
    }
}

/// @brief When the player lands on the pets stop, they spin for a random pet
/// @param player The player gaining the pet
/// @param pets the array of pets
/// @param petList the pet array size
void getPets(Player &player, Pet *pets, int &petList){
    cout << "You have landed on the Pets STOP! Spin (Enter) for a pet!" << endl;
    cout << ">";
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    int roll = (rand() % petList);
    while (pets[roll].getBool()){
        roll = (rand() % petList);
    }
    cout << "You got a " << pets[roll].getName() << "!" << endl;
    // Add pet card to Player's Card Vector then set ownership to true
    player.addCard(std::make_unique<Pet>(pets[roll]));
    pets[roll].setTrue();
}

/// @brief When a player lands on an action space, show the event and pay the
player the affect
/// @param player The player
/// @param actArr The Action Card Array
/// @param actSize The Action Card Array size
void action(Player &player, Action *actArr, int actSize){
    cout << "You have landed on an action space!" << endl;
    player.addA();
    int act = rand() % actSize;
```

```
    // Loop until an unowned card is selected
    while (actArr[act].getBool()) {
        act = rand() % actSize;
    }
    // Add the card to the Player's vector, set the ownership to true, display
card's affect
    player.addCard(std::make_unique<Action>(actArr[act]));
    actArr[act].setTrue();
    cout << actArr[act].getE() << endl;

    player.addMny(actArr[act].getAff()); // Add or subtract money from player
    if (player.getMny() < 0) {
        player.setMny(0); // Ensure money does not go below zero
    }
    cout << "You now have " << player.getMny() << "K" << endl;
}

/// @brief Run at the end of the game to give them their extra bits of money as a
boost
/// @param players The player array
/// @param houses The house cards array
/// @param pSize Player array size
void retire(Player *players, House *houses, int pSize) {
    cout << "=========================================" << endl;
    for (int i = 0; i < pSize; i++){
        cout << "Retiring " << players[i].getName() << "..." << endl;
        if (players[i].getKids() > 0) { // Check for kids
            players[i].addMny(players[i].getKids() * 100);
        }
        if (players[i].getVC() > 0) { // Give player 100 per vacation counter
            players[i].addMny(players[i].getVC() * 100);
        }
        if (players[i].getAC() > 0) { // Give player 100 per action card
            players[i].addMny(players[i].getAC() * 100);
        }
        sellH(players[i]);

    }
}

/// @brief Function used in retire function to help players sell their homes for
extra money (like an investment)
/// @param player The player selling their homes
```

```
void sellH(Player &player) { // Sell House Function
    House *hcpy = player.getH();
    bool soldAny = false;
    for (int j = 0; j < 5; j++) {
        if (hcpy[j].getBool()) { // Check if the house is actually owned
            int roll = spin();
            cout << "Spin (Enter) to sell your house: " << hcpy[j].getName() <<
"! (Odds = Low sell, Even = High Sell)" << endl;
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            if (roll % 2 == 0) {
                cout << "You rolled even! +" << hcpy[j].getS2() << "k!" << endl;
                player.addMny(hcpy[j].getS2());
            }
            else {
                cout << "You rolled odd! +" << hcpy[j].getS1() << "k!" << endl;
                player.addMny(hcpy[j].getS1());
            }
            // Mark house as sold
            hcpy[j] = House(); // Reset to default/empty house
            soldAny = true;
        }
    }
}

/// @brief Prompt a player to buy 1 of 2 houses when landing on a house spot
/// @param player The player buying houses
/// @param arr The House Cards Array
/// @param size The size of the House Array
void buyH(Player &player, House *arr, int size){
    House* cpyArr = player.getH();
    int id1 = rand() % size, id2 = rand() % size;
    char pick;

    while (arr[id1].getBool()) {
        id1 = rand() % size;
    }

    while (arr[id2].getBool() || id2 == id1){
        id2 = rand() % size;
    }

    // Usage of Copy Constructor
    House choice1 = arr[id1];
```

```
    House choice2 = arr[id2];

    do {
        cin.clear();
        pick = '\0';
        cout << "========================================" << endl;
        cout << "Please select the House you want to buy." << endl;
        cout << "(Enter 0 to cancel)" << endl;
        cout << "House 1: " << endl;
        cout << choice1;
        cout << "----------------------------------------" << endl;
        cout << "House 2: " << endl;
        cout << choice2;
        cout << ">"; cin >> pick;
        if (pick == '1'){
            if (player.getMny() < arr[id1].getCost()) { // Check if player has
enough money, kick em out if they don't
                cout << "You do not have enough money. Don't try to buy a house
you can't afford loser" << endl;
                return;
            }
            // Set Card ownership to true -> Add House Card to player's vector of
cards - > validate if player has enough room
            arr[id1].setTrue();
            player.addCard(std::make_unique<House>(choice1));
            bool added = false;
            for (int i = 0; i < 5; i++){
                if (!cpyArr[i].getBool()) { // Use ownership flag instead of name
check
                    cpyArr[i] = choice1;
                    cpyArr[i].setTrue(); // Mark as owned
                    added = true;
                    break;
                }
            }
            if (!added) {
                cout << "You have the maximum amount of houses. House was not
purchased." << endl;
                return;
            }
            cout << "You bought the " << arr[id1].getName() << "!"<< endl;
        }
        else if (pick == '2'){
            if (player.getMny() < arr[id2].getCost()) {
```

```
                cout << "You do not have enough money. Don't try to buy a house
you can't afford loser" << endl;
                return;
            }
            // Set Card ownership to true -> Add House Card to player's vector of
cards - > validate if player has enough room
            arr[id2].setTrue();
            player.addCard(std::make_unique<House>(choice2));
            bool added = false;
            for (int i = 0; i < 5; i++){
                if (!cpyArr[i].getBool()) { // Use ownership flag instead of name
check
                    cpyArr[i] = choice2;
                    cpyArr[i].setTrue(); // Mark as owned
                    added = true;
                    break;
                }
            }
            if (!added) {
                cout << "You have the maximum amount of houses. House was not
purchased." << endl;
                return;
            }
            cout << "You bought the " << arr[id2].getName() << "!"<< endl;
        }
        else if (pick == '0'){
            cout << "You have chosen to not buy a house." << endl;
            return;
        }
        else {cout << "Please enter a valid response (0, 1 or 2)" << endl;}
    } while ((pick != '1') && (pick != '2') && (pick != '0'));

    cout << "========================================" << endl;
    player.setH(cpyArr);
    cin.ignore();
}

/// @brief Prompt a player to pick a new career (if wanted) and change their
career according to so
/// @param player The player being prompted
/// @param carArr The regular Career Cards Array
/// @param colArr The College Career Cards Array
/// @param jSize The size of both of the Career Arrays
void nightSc(Player &player, Career *carArr, Career *colArr, int jSize){
```

```
    char pick;
    cout << "Would you like to go to night school? (y/n)" << endl;
    cin >> pick;
    if (pick == 'y' || pick == 'Y') {
        if (!player.getCol()) {player.setCol();}
        player.rmvCard<Career>();
        player.getCar().setFlse();
        job(player, carArr, colArr, jSize);
    }
    else {
        cout << "You have decided to not go to night school." <<endl;
    }
    cin.ignore();
}

/// @brief Prompt the user a job select depending on whether they wanted to go to
college or not
/// @param player The player being prompted
/// @param carArr The Regular Career Cards Array
/// @param colArr The College Career Cards Array
/// @param jSize The size of the Career Card Arrays
void job(Player &player, Career *carArr, Career *colArr, int jSize) {
    if(player.getCol()){
        jobSlct(player, colArr, jSize);
    }
    else{
        jobSlct(player, carArr, jSize);
    }
}

/// @brief Prompt the user to pick between 1 of 2 career choices
/// @param player The player being prompted
/// @param arr The Career Card Array
/// @param size The size of the Career Array
void jobSlct(Player &player, Career *arr, int size){
    int id1 = rand() % size, id2 = rand() % size;

    while(arr[id1].getBool()){
        id1 = rand() % size;
    }

    while(arr[id2].getBool() || id2 == id1){
        id2= rand() % size;
```

```
    }

    // Usage of Copy Constructor
    Career choice1 = arr[id1];
    Career choice2 = arr[id2];

    char pick;
    cout << "========================================" << endl;
    cout << "Pick one of the two careers: " << endl;
    cout << "Job 1: " << endl;
    cout << choice1;
    cout << "Job 2: " << endl;
    cout << choice2;
    cout << "Enter 1 or 2 in accordance to the job:" << endl;

    do {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        pick = '\0';
        cin >> pick;
        if (pick == '1') {
            player.setCar(choice1);
            player.addCard(std::make_unique<Career>(choice1));
            arr[id1].setTrue();
            cout << "You have chosen: " << choice1.getTi() << endl;
            cout << "========================================" << endl;
        }
        else if (pick == '2') {
            player.setCar(choice2);
            player.addCard(std::make_unique<Career>(choice2));
            arr[id2].setTrue();
            cout << "You have chosen: " << choice2.getTi() << endl;
            cout << "========================================" << endl;
        }
        else {cout << "Please enter a valid response (1 or 2)" << endl;}
    } while((pick != '1') && (pick != '2'));
}
```